

Abschluss-Testat: ROM-Speicher & Automatisierte Testbench

Ausgangslage:

Der Digitalschaltungs-Simulator funktioniert hervorragend. Bisher müssen Nutzer jedoch für komplexe Funktionen (wie z. B. einen 7-Segment-Decoder) dutzende AND- und OR-Gatter mühsam von Hand verschalten. In der Industrie nutzt man stattdessen oft einen ROM (Read-Only Memory) als Look-Up Table (LUT). Die Wahrheitstabelle wird einmalig aus einer Datei geladen, und das Gatter liefert den passenden Wert zu den anliegenden Eingängen.

Anleitung Repository-Setup & Abgabe

Damit Ihre Prüfungsleistung sicher und vor den Blicken anderer geschützt eingereicht wird, nutzen wir private Repositories. Da der Start-Code als GitHub-Template bereitgestellt wird, ist die Einrichtung in wenigen Klicks erledigt.

Bitte führen Sie direkt zu Beginn des Testats diese Schritte aus:

Schritt 1: Ein eigenes, privates Repository aus dem Template erstellen

1. Öffnen Sie die Vorlage im Browser: <https://github.com/dantschi/digisim>
2. Klicken Sie oben rechts auf den grünen Button "Use this template" und wählen Sie "Create a new repository".
3. Vergeben Sie unter "Repository name" einen eindeutigen Namen (z. B. digisim-testat-max-mustermann).
4. **WICHTIG:** Wählen Sie bei der Sichtbarkeit zwingend "Private" aus! (Ist Ihr Code öffentlich, können andere abschreiben).
5. Klicken Sie auf den grünen Button "Create repository". GitHub kopiert nun den gesamten Start-Code in Ihr neues, privates Projekt.

Schritt 2: Den Prüfer einladen Damit Ihr Code am Ende bewertet werden kann, müssen Sie Leserechte auf Ihr privates Repo vergeben.

1. Gehen Sie in Ihrem neuen Repository oben auf den Reiter "Settings".
2. Klicken Sie links im Menü auf "Collaborators". (Ggf. müssen Sie hier Ihr Passwort bestätigen).
3. Klicken Sie auf den grünen Button "Add people".
4. Suchen Sie nach dem GitHub-Nutzernamen des Dozenten (dantschi) und fügen Sie ihn hinzu.

Schritt 3: Code klonen und in VS Code öffnen Laden Sie nun Ihren Code auf Ihren Rechner und starten Sie Ihre Entwicklungsumgebung. Öffnen Sie Ihr Terminal (oder die Git Bash) und führen Sie folgende Befehle aus:

1. Klonen Sie IHR NEUES Repository (nicht das Template!): Ersetzen Sie die URL durch den Link zu Ihrem soeben erstellten Repository: `git clone https://github.com/IHR-USERNAME/IHR-NEUES-REPO-NAME.git`

2. Wechseln Sie in den soeben heruntergeladenen Ordner: `cd IHR-NEUES-REPO-NAME`

3. Öffnen Sie das Projekt direkt in Visual Studio Code: `code .` (Alternativ: Öffnen Sie VS Code manuell und gehen Sie auf "File" -> "Open Folder...", um den geklonten Ordner auszuwählen).

Schritt 4: Entwickeln, Sichern & Abgeben Während des Testats arbeiten Sie ganz normal in Ihrem lokalen Ordner. Wenn Sie fertig sind (und die Datei `REPORT.md` ausgefüllt haben), sichern Sie Ihre Arbeit wie gewohnt auf GitHub:

```
git add . git commit -m "Testat finale Abgabe" git push
```

Die finale Abgabe: Reichen Sie die URL zu Ihrem GitHub-Repository per Moodle ein. Da Sie den Dozenten in Schritt 2 eingeladen haben, kann der Code von ihm eingesehen werden.

WICHTIG: Neben ihrem Code gehört zur Abgabe eine Datei namens `REPORT.md`. Sie existiert bereits im Startcode – enthält aber noch nicht die erforderlichen Inhalte. (Details siehe Teil 3).

Teil 1: Das ROM-Gatter & Parser (Pflichtaufgabe zum Bestehen)

Erweitern Sie den Simulator um ein ROM-Modul, das seine Logik aus einer Datei bezieht. Gehen Sie dabei exakt nach folgendem Bauplan vor:

1. Die Klasse `RomGate`

- Erstellen Sie eine neue Klasse `RomGate`, die von `Gate` erbt.
- Der Konstruktor: Erweitern Sie den Konstruktor so, dass er den Dateipfad zur ROM-Datei (z. B. `"decoder.rom"`) entgegennimmt.
- Datei einlesen: Öffnen Sie direkt im Konstruktor einen `std::ifstream` für diese Datei. Die Datei enthält pro Zeile genau ein Zeichen (eine `'0'` oder `'1'`). Lesen Sie die Datei zeilenweise ein und speichern Sie die Werte als Wahrheitswerte (`true/false`) in einem Klassen-Attribut (z. B. `std::vector<bool> m_truthTable;`).
- Hinweis: Das Einlesen darf auf keinen Fall in der `evaluate()`-Methode passieren, da Festplattenzugriffe in jedem Taktzyklus den Simulator extrem verlangsamen würden!

2. Die Logik (`evaluate()`)

In dieser Methode werten Sie die Eingänge aus und suchen den passenden Wert in der Wahrheitstabelle:

- Wenden Sie das Pull-Prinzip auf alle verbundenen Inputs an (denken Sie an die DFS-Tiefensuche und das Cache-Flag!).
- Wandeln Sie die Eingänge in eine Ganzzahl (den Array-Index) um.
 - Tipp zur Mathematik: Nutzen Sie eine Schleife über `m_inputs`. Wenn Input 0 auf true steht, addieren Sie $2^0=1$ zum Index. Wenn Input 1 auf true steht, addieren Sie $2^1=2$, bei Input 2 addieren Sie $2^2=4$, usw.
- Setzen Sie `m_output` auf den Wert aus `m_truthTable[index]`.

3. Integration in den Parser (GateFactory)

- Ändern Sie die GateFactory und die Engine so ab, dass nun drei Parameter aus der .circuit-Datei gelesen werden können (Befehl, Typ, Name und optional Dateipfad).
- Die Engine muss nun diesen Befehl fehlerfrei parsen können: GATE ROM meinDecoder data/decoder.rom
- Erstellen Sie abschließend eine kleine .rom-Datei und eine .circuit-Datei, in der Sie zwei Schalter an das ROM-Gatter anschließen. Weisen Sie die Funktion über Konsolenausgaben (oder die ASCII-Waveform) nach.

Teil 2: Die Testbench / AssertGate (Zusatzaufgabe)

Für eine sehr gute Note beweisen Sie, dass Sie Software-Qualitätssicherung verstanden haben. Entwerfen Sie eigenständig ein Bauteil zur automatisierten Schaltungsüberprüfung (Testbench).

Anforderungen:

1. Entwerfen Sie eine Klasse AssertGate.
2. Das Bauteil muss über die .circuit-Datei instanziiert und verkabelt werden können. Es nimmt genau zwei Signale entgegen: Einen Ist-Wert (aus der Schaltung) und einen Soll-Wert (als Referenz).
3. Implementieren Sie die Überprüfungslogik. Stimmen beide Werte überein, verhält sich das Gatter passiv. Weichen die Werte ab, muss die Simulation sofort hart abgebrochen werden (z. B. durch Werfen einer Exception mit einer detaillierten Fehlermeldung auf der Konsole).
4. Erstellen Sie eine modifizierte .circuit-Datei, die diesen Fehlerfall absichtlich provoziert, um Ihre Testbench in Aktion zu zeigen.

Teil 3: Das Abgabedokument (REPORT.md)

Im Hauptverzeichnis Ihres Repositories liegt eine Datei namens REPORT.md. Kopieren Sie die folgenden drei Blöcke in die Datei und beantworten Sie die Fragen ausführlich und spezifisch zu Ihrem geschriebenen Code.

Block A: KI-Protokoll (AI Transparency Log)

Der Einsatz von KI-Tools (GitHub Copilot, ChatGPT, etc.) wird begrüßt, soll aber nicht unreflektiert geschehen.

1. Welche Werkzeuge haben Sie für diese Aufgabe genutzt?
2. Fassen Sie Ihren KI-Arbeitsprozess zusammen:
 - Wie haben sie ihre Prompts formuliert?
 - Kurze Beschreibung der Anpassungen: Was mussten Sie am KI-Code oder am Prompt ändern, damit er funktioniert? (z.B. "KI hat Header vergessen", "Variable umbenannt", "Modulo-Logik korrigiert").

Block B: Architektur-Verständnis

1. Warum ist es aus Architektursicht ein schwerwiegender Fehler, wenn die `std::ifstream`-Logik zum Einlesen der `.rom`-Textdatei in der `evaluate()`-Methode statt im Konstruktor platziert wird?
2. Sie haben das `RomGate` zur `LogicEngine` hinzugefügt, ohne die eigentliche Simulationsschleife (`LogicEngine::doTick()`) umschreiben zu müssen. Welches objektorientierte Prinzip aus der Vorlesung macht das in C++ möglich und wie funktioniert der Mechanismus intern?
3. In unserem Code nutzen wir für die Engine `std::unique_ptr` und für die Kabel `Gate*` (Raw Pointers). Erklären Sie das Ownership-Konzept dahinter. Wer ist für das Löschen des Speichers verantwortlich und warum verhindert diese Trennung Memory-Leaks?

Block C: Edge-Cases (Fehlerbehandlung)

1. Erklären Sie präzise, wie Ihr Code reagiert, wenn der Nutzer in der `.circuit`-Datei ein `RomGate` anlegt, aber der angegebene Pfad zur `.rom`-Datei falsch ist (Datei existiert nicht). Auf welche Code-Zeile in Ihrem Projekt verlassen Sie sich in diesem Moment?
2. Was passiert in Ihrem `evaluate()`, wenn der Nutzer 3 Kabel an das `RomGate` anschließt (maximaler Index = 7), Ihre eingelesene Wahrheitstabelle aber versehentlich nur 4 Zeilen lang ist? Wie schützt Ihr Code das Programm vor einem Absturz (Segmentation Fault)?